B3 in plain language: a team briefing

Vidhi Kothari · Challenge B3: "It Is Running, but Is It Still Working?" · For Lauren, Anastasiya and Stephen, before the presentation

1. The question in one sentence

Our API can be up, answering every request quickly, and still be giving bad answers. B3 asks: **how do we tell "the system is up" apart from "the AI is still doing its job"**, and what should happen when something changes?

The one thing to remember: a healthy service does not mean the AI is behaving appropriately. And not every change in the data matters. What matters is whether the change reaches the model's output and the alerts people receive.

2. What I found (the diagnosis)

- **The service is healthy.** In every test: 0 API errors, the model is loaded, the health check passes.
- **But the model's behavior is questionable.** It raises an alert on 45.9% of rows, while only 7.3% are real spikes. Precision at the 0.5 alert threshold is 6.4%: about 1 alert in 16 is a real spike.
- **Its ranking quality is near the baseline.** PR-AUC is 0.070 vs a prevalence baseline of 0.073. In plain words: on the data we have, we cannot show it ranks spikes better than the no-information baseline. That does *not* prove it is useless; our data are short and effectively the same data it was trained on.
- **Why:** the model's score mostly follows **trade intensity** (how many messages arrived in the last 60 seconds). Score vs trade intensity: r = 0.80. Score vs actual spikes: r = 0.006, essentially no relationship.
- **A data quirk matters a lot.** The recording has a 171-second gap. Trade intensity is a rolling 60-second count, so after the gap it starts low and refills. The model sees that refill as a big change. (The message rate itself is about the same on both sides: 4.92 vs 4.74 messages per second.)

3. The hypothesis, and what changed since the interim

The core hypothesis was already in our interim, and it survived: health checks and raw drift tests alone can't tell you whether the AI still works. Monitor serving health, data/prediction drift and labeled performance separately. A change in the input data should trigger **investigation**, not automatic retraining.

What changed was how we read one piece of evidence. At the interim, we compared the first half of the recording with the second half and treated it as a "nothing changed" test, so the drift flags looked like a false alarm. Later measurement showed the halves really differ (because of the gap and the refill): the model's alert rate goes from 13.5% to 78.3% while real spikes are 6.1% vs 8.5%. So that comparison became a **real-change** scenario (S2), and we built a new "nothing changed" test (S0). We revised the experiment rather than forcing the original explanation to fit.

4. What I built (the intervention)

One script, `monitoring/b3_monitor.py`, that checks three separate things:

Layer	What it asks	What it says
Tier 1: serving status	Is the service up and producing predictions?	HEALTHY / DEGRADED (some requests failed) / UNAVAILABLE. Only UNAVAILABLE pages someone (PAGE).
Tier 2: drift, graded by impact	Did the data change, and did it reach the model's output and the alert volume?	OK / LOW / MEDIUM / HIGH (see below)
Tier 3: model performance	Once the true answers arrive, is the model better than the baseline?	A separate, after-the-fact report. It never sets the severity.

The severity ladder in plain words: input data changed but the model didn't react = **LOW** (data-quality ticket). The model's output moved, or alerts moved moderately = **MEDIUM** (investigate). The alert volume is seriously disrupted = **HIGH** (escalate). The exact cutoffs (5 percentage points, 20 percentage points, 2x) are project-specific operating thresholds, not industry standards.

5. How I tested it (the evidence)

Four controlled scenarios, each comparing the original recording with a changed version. The "before" column is what the supplied drift tool (Evidently, using the KS test) says; the "after" column is my monitor.

Scenario	What it is	Before: features flagged	After: monitor	What happened to alerts
S0	Same data, split odd/even rows (nothing changed)	0 of 4	OK	46.2% to 45.7%
S1	Supplied "drifted" file: wider bid/ask spread	1 of 4	LOW	45.9% to 46.2% (barely moved)
S2	First half vs second half (gap + refill)	3 of 4	HIGH	13.5% to 78.3%
S3	Every other message dropped, replayed through Kafka	2 of 4	HIGH	45.9% to 1.8%, service still HEALTHY

The key comparison: S1 and S2 both show statistical drift. But in S1 the model's output and alert workload barely move, while in S2 alerts jump from 13.5% to 78.3%. That is why raw drift alone is not enough for an operational decision. S3 adds a second example: the service stays HEALTHY while alerts collapse from 45.9% to 1.8%.

6. How much to trust it

- **What we know:** across 1,485 tested monitoring-setting combinations, the S0-S3 ordering remained unchanged. This is **internal threshold robustness, not independent validation**: it shows the result doesn't hinge on the exact cutoffs I picked.
- **What we don't know:** all scenarios come from one ~5-minute recording, which also shaped the design. The training file is the same recording repeated. There is no independent held-out test, so we don't know how the monitor or the model behave on genuinely new data.

7. What I recommend

- Use the monitor provisionally, in a limited way, until it is tested once on an independently recorded replay window.
- Do not automatically retrain because input data drifted (like S1); route it to a data-quality check.
- Re-evaluate the model on genuinely out-of-sample labeled data before changing thresholds, retraining or replacing it.

8. How B3 connects to the rest of the team

These are integration points as described in the team's materials; B3 did not independently test B1 or B2 work. **B2** concerns /health reporting whether the model is loaded; B3 adds that a loaded model is not necessarily behaving appropriately. **B1** concerns keeping up with the feed; in B3's scenarios, breaks or losses in the message stream changed trade intensity, which drives the model's predictions.

9. My slides at a glance (about 3 minutes)

Slide	Message	Time
1. It's running. Is it still working?	Healthy service, questionable behavior; the interim hypothesis survived	~30 s
2. Diagnosis (Figure A)	Alerts follow trade intensity, not spikes; the split-half correction	~45-50 s
3. Intervention and evidence (Figure B)	The tiers; S1 (LOW) vs S2/S3 (HIGH)	~60-70 s

4. How far to trust it	What we know / don't know / recommend	~35-40 s

Backups: full severity ladder and Tier 1 statuses; Tier 3 lift intervals (Figure C); where each setting comes from; AI supervision. If a question goes to one of these, hand it to me.

10. Q&A: questions we might get, with short answers

If you get one of these during the team Q&A, the short answer below is safe to give; for anything deeper, pass it to me.

Q: Your interim called the split-half a false alarm. Was the interim wrong?

The core hypothesis survived; our interpretation of that one comparison did not. The rolling-window mechanism was real, but the alerts really changed (13.5% to 78.3%), so it was not a false alarm. We made it S2 and built a new no-change control (S0). The correction actually supports the hypothesis: KS flagged both S1 and S2, and only the model-impact view tells them apart.

Q: Why not just retrain whenever drift is detected?

Not all drift matters. S1 has a huge input shift but the model barely reacts. Labeled evidence is needed to establish a performance problem, and root-cause analysis determines whether retraining is appropriate.

Q: Why use PSI as well as the KS test?

The KS p-value shows evidence that distributions differ, not how much that matters. SciPy's KS test assumes independent samples, and our streaming rows are serially dependent, so we don't read the p-values as calibrated. PSI measures how big a shift is; applied to the model's score and alert rate, it shows whether the shift reaches the model's behavior. KS stays as the "before" comparison.

Q: Where do the PSI cut-offs 0.10 and 0.25 come from?

They are the Lewis constants, traced through Potgieter et al. (2026): conventional credit-scorecard heuristics, not calibrated thresholds. Feature-engine currently defaults to 0.25 and documents 0.10 and 0.25 as commonly used values. In our monitor only 0.10 affects severity; 0.25 is just a reference line.

Q: Where do 5 pp, 20 pp and 2x come from?

They were proposed by the AI assistant and accepted by Vidhi as project-specific settings; they are not external standards, and no production service-level target was supplied. That's why we tested a range of alternatives.

Q: 1,485 out of 1,485 sounds like proof. Is it?

No. It reran the same four scenarios from the same recording with different settings. It shows the result doesn't depend on the exact cutoffs; it does not show the monitor works on new data. Internal threshold robustness, not independent validation.

Q: Why is there no held-out test?

The repository has one genuine recording; the training file is that recording repeated. Re-splitting it would not give independent data. The next step is to test the frozen monitor once on a newly recorded replay window.

Q: Isn't S3 just synthetic?

S3 is pipeline-faithful: message loss was injected at the raw-message level, replayed through Kafka, and recomputed by the real featurizer. But pipeline fidelity is not the same as external validity: it is still a constructed fault on the same recording, not a real incident.

Q: Why doesn't the model-performance check (Tier 3) set the severity?

The same Tier 3 outcome appears in every window, including the no-change control S0, so using it as a severity would make every scenario HIGH and the monitor could no longer tell situations apart. Labels also arrive late, and the intervals are exploratory.

Q: What's the difference between DEGRADED and PAGE?

DEGRADED: the service is up and some predictions succeed, but some requests fail. It is reported with error counts but doesn't page anyone, because no service-level target was supplied. UNAVAILABLE means no predictions at all, and that pages on-call (PAGE). Neither happened in our saved runs; they were checked only with constructed inputs.

Q: What does "above-baseline performance not established" mean? Is the model useless?

On the available data, we do not establish PR-AUC performance above the prevalence (no-information) baseline. The exploratory interval includes 0, but the rows are serially dependent and not independent out-of-sample data, so we don't treat that as a formal test. This does not prove the model is useless or incapable of performing better on unseen data.

Q: Why does the model alert so often?

Its alerts follow trade intensity. The supplied model uses class_weight='balanced', which assigns class weights inversely proportional to class frequency during fitting. We have not tested whether this setting caused the high alert rate.

Q: What should be tested next?

First, test the frozen monitor once on an independently recorded replay window. Second, evaluate the model on genuinely out-of-sample labeled data. After that, test real DEGRADED/UNAVAILABLE serving faults and investigate whether the balanced class weighting contributes to the unusually high alert rate.

Q: Did AI come up with B3?

No. The hypothesis was in our interim. The AI assistant contributed specific explanations and design proposals, and some were rejected after measurement, including its explanation of the split-half drift. Details are in B3_AI_DECISIONS.md.

Words that might come up

Term	Plain meaning
Drift	The data (inputs) or the model's outputs look different from before.
PSI	A number for how much a distribution moved. Bigger = bigger shift.
KS test / p-value	A statistical test of whether two samples differ; it says nothing about whether the difference matters.
PR-AUC / prevalence	How well scores rank real spikes above non-spikes / the score a no-information ranking would get.
Trade intensity	Number of messages in the last 60 seconds; low right after a break, then refills.
Alert rate	Share of rows where the model raises a spike alert (score 0.5 or more).

Full detail: `B3_Explainer.pdf`, `B3_README.md`, `B3_AI_DECISIONS.md` on branch `B3---Vidhi`.